Process & Decision Documentation

Project/Assignment Decisions

For this week's sidequest, I used Week 6's completed sidequest code and added a debug screen as per sidequest requirements. Another decision I made was to finally complete it with sounds. Last week, I couldn't figure out how to add sounds, so I took some more time with it this week to finally be able to incorporate. However, I still couldn't figure out why my hitEnemy doesn't make a sound.

Role-Based Process Evidence

Name: Queene Calading

Role(s): Designer

Primary responsibility for this work: Implementing debug screen and sounds

Goal of Work Session:

- Find and incorporate the new assets and debug into the playable code

Tools, Resources, or Inputs Used:

ChatGPT 5.2 (Code help, debugging)

VSCode

Github

Prior starter code given (W6 Example 5)

```
131   soundManager = new SoundManager();
132+      You, 9 hours ago • copy week 6
133+  soundManager.load("jump", "assets/sfx/jump.wav");
134+  soundManager.load("hitEnemy", "assets/sfx/hitEnemy.wav");
135+  soundManager.load("leafCollect", "assets/sfx/leafCollect.wav");
136+  soundManager.load("receiveDamage", "assets/sfx/receiveDamage.wav");
137+  soundManager.load("music", "assets/sfx/music.wav");
138+
139+
140+ const music = soundManager.sfx["music"];
141+   music.setLoop(true);
142+   music.setVolume(0.2);
143+
132 144
133 145   // --- Parallax layer defs (VIEW) ---
134 146   const defs = levelPkg.level?.view?.parallax ?? [];
```

```
TIMELINE   main.js
add sounds   Queene C   3 mins
File Saved              16 mins
File Saved
File Saved              17 mins
File Saved              17 mins
add debug overlay   Queene C   31 mins
File Saved              36 mins
File Saved              38 mins
copy week 6   Queene C   10 hrs
```

GenAI Documentation

Date Used: March 23, 2026

Tool Disclosure: ChatGPT 5.2

Purpose of Use: Coding guidance in how to add sounds, debug screen and debugging code

Summary of Interaction:

ChatGPT helped by providing step-by-step instructions in implementing the new assets (sounds). It also helped in debugging my issues with the sounds to why it wasn't working.

Human Decision Point(s):

- Accepted debugging guidance and structural code suggestions, but adjusted parameters to match naming conventions etc. (soundManager, when it should just say sound). Rejected other suggestions in extra debugging add ons, felt confused / too complicated to implement

Integrity & Verification Note:

I verified that the given ideas were compatible with p5.js physics and drawing by checking the online p5.js library of what each parameter means in a line of code, and ensured no AI-generated code was copied without understanding (I asked questions and made it so that I was using AI to show me how / what I could do, rather than telling it what needs to be done and copy paste). I also adjusted values based on playtesting results

Scope of GenAI Use:

GenAI contributed to debugging guidance and example implementations, but didn't just make the code for me without understanding. Implementations to the actual code, with edits are done by me.

Limitations or Misfires:

- Some AI code examples included unnecessary complexity for my skill level, so I simplified them to maintain readability and ensure I fully understood each mechanic
- It couldn't debug what was wrong with the hitEnemy sound (doesn't work)

Appendix

Transcript

Prompt:
(Inserted the example 6 code as seen in link)
https://chatgpt.com/share/69b44ae3-2fe4-8004-9617-89d6b84c8ecc
Here is the side quest for this week (ignore the one for Week 6 in the Side Quest assignment outline): Find a free sprite sheet online (make sure to provide credit in your README) and change the characters for the player, leaf, and enemy. Find a free tile set online (make sure to provide credit in your README) and change the look of the level. Add sounds for jumping, attacking, taking damage, and collecting objects, and a looping background track. How do I go about changing the code to incorporate my own assets

Response:

Your codebase is already **very modular**, which is good — you mostly **won't change gameplay code**. You'll mainly update:

1. **Asset loading**
2. **Sprite/animation definitions**
3. **Tile graphics**
4. **Sound loading + event wiring**
5. **Parallax/background assets (optional)**

I'll walk through exactly **where in your architecture to change things**.

---

1. Replace Player / Enemy / Leaf Sprite Sheets

Your entities are created inside:

- `src/entities/PlayerEntity.js`
- `src/world/BoarSystem.js`
- `src/world/TileBuilder.js` (leafs)

The **sprites themselves come from `assets`**, which are loaded in:

src/AssetLoader.js

So **the first step is replacing assets in the loader.**

---

2. Edit `AssetLoader.js`

Your loader probably looks something like this (simplified):

```
assets.player = loadSpriteSheet("assets/player.png", {
 frameW: 32,
 frameH: 32
});
```

To use a new sprite sheet:

Example

Suppose your sheet is:

assets/player_knight.png

Each frame:

64 x 64

Update:

```
assets.player = loadSpriteSheet("assets/player_knight.png", {
 frameW: 64,
 frameH: 64
});
```

---

3. Update Animations

Your **PlayerEntity** probably has something like:

```
this.sprite.addAni("idle", assets.player, 0, 3);
this.sprite.addAni("run", assets.player, 4, 9);
this.sprite.addAni("jump", assets.player, 10, 13);
this.sprite.addAni("attack", assets.player, 14, 17);
```

You must update these to match your new sheet layout.

Example:

```
this.sprite.addAni("idle", assets.player, 0, 5);
this.sprite.addAni("run", assets.player, 6, 11);
this.sprite.addAni("jump", assets.player, 12, 15);
this.sprite.addAni("attack", assets.player, 16, 21);
```

---

4. Replace Enemy (Boar)

Boars are built in:

src/world/BoarSystem.js

Look for something like:

```
e.addAni("walk", assets.boar, 0, 5);
e.addAni("death", assets.boar, 6, 10);
```

Replace with your enemy sprite:

assets/slime.png

Example:

```
e.addAni("walk", assets.slime, 0, 3);
e.addAni("death", assets.slime, 4, 7);
```

And update loader:

```
assets.slime = loadSpriteSheet("assets/slime.png", {
 frameW: 32,
 frameH: 32
});
```

---

5. Replace Leaf Collectible

Leafs come from **TileBuilder**.

Find something like:

```
this.leaf = new Group();
this.leaf.img = assets.leaf;
```

Then update the asset in `AssetLoader.js`:

```
assets.leaf = loadImage("assets/gem.png");
```

or animated collectible:

```
assets.gem = loadSpriteSheet("assets/gem.png", {
 frameW: 16,
 frameH: 16
});
```

---

6. Replace Tileset

Tiles are defined in:

data/levels.json

Your tile characters map to sprites in:

src/world/TileBuilder.js

Example:

```
this.ground.img = assets.ground;
this.platformsL.img = assets.platform;
```

Change assets:

```
assets.ground = loadImage("assets/tiles/grass.png");
assets.platform = loadImage("assets/tiles/platform.png");
assets.wall = loadImage("assets/tiles/rock.png");
```

Your **tile size must match your level config**:

From `levels.json`:

```
tileW: 24
tileH: 24
```

If your tileset uses **16x16** or **32x32**, you must update:

tileW
tileH

in `levels.json`.

---

7. Add Sounds

You already have a **SoundManager**, which is great.

Step 1: Load sounds

Inside `main.js` after creating `soundManager`:

```
soundManager.load("jump", "assets/audio/jump.wav");
soundManager.load("attack", "assets/audio/attack.wav");
soundManager.load("hurt", "assets/audio/hurt.wav");
soundManager.load("leaf", "assets/audio/collect.wav");
soundManager.load("bgm", "assets/audio/music.mp3");
```

Step 2: Hook events

You already have these:

In `Game.js`:

```
this.events.on("leaf:collected", () => this.sound.play("leaf"));
this.events.on("player:damaged", () => this.sound.play("hurt"));
this.events.on("boar:damaged", () => this.sound.play("hit"));
this.events.on("level:won", () => this.sound.play("win"));
```

Add:

```
this.events.on("player:jump", () => this.sound.play("jump"));
this.events.on("player:attack", () => this.sound.play("attack"));
```

Step 3: Emit events

Inside `PlayerController`:

Jump:

```
this.events.emit("player:jump");
```

Attack:

```
this.events.emit("player:attack");
```

---

8. Add Looping Background Music

In `main.js` after loading:

```
soundManager.sfx.bgm.setLoop(true);
soundManager.play("bgm");
```

Or create a music channel.

---

9. Add Asset Credits

In your **README.md**:

Example:

```
ASSET CREDITS

Player sprites:
Knight character by FooBar Studios
https://opengameart.org/knight-character

Enemy sprites:
Slime enemy by Jane Smith
https://itch.io/slime-pack

Tileset:
Fantasy Tileset by Pixel Frog
https://itch.io/fantasy-tiles

Sound effects:
Jump/Attack sounds from Kenney.nl
https://kenney.nl/assets/audio

Music:
Forest Theme by Eric Matyas
https://soundimage.org
```

---

10. Where You Actually Change Code

You will likely modify **only these files**:

src/AssetLoader.js ← most changes
src/entities/PlayerEntity.js
src/world/BoarSystem.js
src/world/TileBuilder.js
main.js ← sound loading
PlayerController.js ← emit jump/attack events
levels.json ← tile size if tiles change

Everything else stays untouched.

Recommended Free Asset Sites

Good sources for this assignment:

Sprites

itch.io/assets
opengameart.org
kenney.nl
craftpix.net (free section)

Tilesets

kenney.nl
itch.io tilesets
opengameart

Sounds

kenney.nl
freesound.org
opengameart

If you want, I can also show you

- **3 really good sprite packs that fit your game immediately**
- **a perfect tileset for your tilemap**
- **exact animation frame setups**

So you can plug them in **in under 20 minutes.**

my player is cut off, where the body is on top but head is bottom

Ah, got it — that sounds like a **sprite/animation offset problem** rather than a game logic issue. In p5play (or similar sprite systems), the “body upside-down” look usually happens when the **sprite’s origin/anchor** is misaligned. The sprite might be drawing its top-left corner at the player’s position, or the animation frame might be flipped vertically.

Here’s how to troubleshoot:

---

1. Check the sprite’s `ani` / image

- In `PlayerEntity.buildSprites()`, make sure the sprite images are **upright** and not accidentally flipped vertically.
- Example:

this.sprite.mirrorY(false); // ensure not flipped

---

2. Verify sprite’s `ani` anchor/offset

- p5play by default positions a sprite by its **center**, but some custom builds or loaded images might use **top-left**.
- If your sprite image’s “head” is at the bottom of the image file, you’ll see that cut-off effect.

You can adjust the origin:

this.sprite.ani.offsetY = -this.sprite.height/2; // shift sprite up so head aligns

Or explicitly set `center` mode:

this.sprite.center = createVector(this.sprite.width/2, this.sprite.height/2);

---

3. Check player controller position updates

- In `PlayerController.update()`, make sure `sprite.x` and `sprite.y` are **the sprite’s center**, not bottom or top.

- Sometimes collision handling or physics moves the sprite by its top-left, while the image draws from center → misalignment.

---

4. Temporary debugging

- Draw a small dot at `sprite.x, sprite.y` to see where the engine thinks the player is:

```
fill('red');
circle(this.playerCtrl.sprite.x, this.playerCtrl.sprite.y, 4);
```

- If the red dot is near the head, your sprite's image anchor is reversed.

If you want, I can write a **quick patch for your `PlayerEntity`** so the sprite body aligns correctly with the physics/colliders and stops being upside-down.

Do you want me to do that?

yes

Perfect! Based on your `Level.js` and `PlayerEntity.js` setup, the easiest fix is to **ensure the sprite's origin is centered**, and any "y offset" is adjusted so the head sits at the top as expected. Here's a patch you can apply inside your `PlayerEntity.buildSprites()` method:

```
buildSprites() {
 // assuming you already create sprite(s) here
 this.sprite = createSprite(this.x, this.y);

 // attach animations
 for (const [name, aniFrames] of Object.entries(this.anis)) {
   this.sprite.addAnimation(name, ...aniFrames);
 }

 // --- FIX ORIGIN / OFFSET ---
 // 1. Make sprite draw from center
 this.sprite.setCollider("rectangle", 0, 0, this.sprite.width, this.sprite.height);
 this.sprite.mirrorY(false);      // ensure no vertical flip
 this.sprite.rotation = 0;        // just in case

 // 2. Optional: adjust vertical offset if head seems below body
 // Use half-height shift to align correctly
 this.sprite.offset.y = -this.sprite.height / 2;
```

```
 // 3. Ensure physics/collisions use center
 this.sprite.collider = "rectangle";
}
```

Notes:

1. `this.sprite.offset.y` moves the image relative to the sprite's actual physics/collider.
    - Negative values shift it **up**, so the head appears at the top.
    - Positive values shift **down**.
2. Make sure `mirrorY(false)` is called if the sprite was flipped vertically anywhere.
3. Collider should match sprite dimensions; otherwise, collisions may feel off.

💡 **Quick debug trick:**
Draw the sprite's center with a small red circle:

```
fill('red');
circle(this.sprite.x, this.sprite.y, 4);
```

If the red dot is at the middle of the player body, your sprite is now correctly aligned.

If you want, I can also write a **full drop-in replacement for `PlayerEntity.buildSprites()`** that fixes both alignment **and attack hitbox offsets** so the player is fully upright and collides properly.

Do you want me to do that?

Prompt:

```
 buildSprites() { const { frameW = 32, frameH = 32 } = this.tilesCfg; this.sprite = new Sprite(this.startX, this.startY, frameW, frameH); this.sprite.rotationLock = true; const anis = this.assets?.playerAnis; const img = this.assets?.playerImg; if (img) this.sprite.spriteSheet = img; if (anis && typeof anis === "object") { this.sprite.anis.w = frameW; this.sprite.anis.h = frameH; this.sprite.anis.offset.y = this.ANI_OFFSET_Y; this.sprite.addAnis(anis); this._setAni("idle"); } else { this.sprite.img = img; } // collider this.sprite.w = this.COLLIDER_W; this.sprite.h = this.COLLIDER_H; this.sprite.friction = 0; this.sprite.bounciness = 0; // ground sensor (query-only) this.sensor = new Sprite(); this.sensor.x = this.sprite.x; this.sensor.y = this.sprite.y + this.sprite.h / 2; this.sensor.w = this.sprite.w; this.sensor.h = 2; this.sensor.mass = 0.01; this.sensor.removeColliders(); this.sensor.visible = false; const j = new GlueJoint(this.sprite, this.sensor); j.visible = false; return this; } reset() { this.health = this.maxHealth; this.dead = false; this.pendingDeath = false; this.deathAnimStarted = false; this.invulnTimer = 0; this.knockTimer = 0; this.attacking = false;
```

this.attackFrameCounter = 0; this.attackHitThisSwing = false; if (!this.sprite) return; this.sprite.x = this.startX; this.sprite.y = this.startY; this.sprite.vel.x = 0; this.sprite.vel.y = 0; this.sprite.tint = "#ffffff"; this._setAni("idle"); } // ---------- // queries // ---------- isGrounded(solids) { const s = this.sensor; if (!s) return false; // Allow either { ground, platformsL, ... } OR an array [ground, platformsL, ...] const list = Array.isArray(solids) ? solids : Object.values(solids || {}); for (const g of list) { if (g && s.overlapping(g)) return true; } return false; } // ---------- // timers // ---------- tickTimers() { if (this.invulnTimer > 0) this.invulnTimer--; if (this.knockTimer > 0) this.knockTimer--; } // ---------- // actions (mutations) // ---------- stopX() { this.sprite.vel.x = 0; } moveLeft() { this.sprite.vel.x = -this.MOVE_SPEED; this.sprite.mirror.x = true; } moveRight() { this.sprite.vel.x = this.MOVE_SPEED; this.sprite.mirror.x = false; } jump() { this.sprite.vel.y = -1 * this.JUMP_STRENGTH; } startAttack() { this.attacking = true; this.attackHitThisSwing = false; this.attackFrameCounter = 0; this.stopX(); this._playAni("attack", 0); } markAttackHit() { this.attackHitThisSwing = true; } clampToBounds(bounds) { const half = (this.sprite?.w ?? 0) / 2; const maxX = (bounds?.levelW ?? this.sprite.x) - half; this.sprite.x = constrain(this.sprite.x, half, maxX); } // ---------- // animation (visual state) // ---------- applyAnimation({ grounded, won }) { if (!this.sprite?.anis || Object.keys(this.sprite.anis).length === 0) return; // ---- DEAD: play once, then hold last frame (prevents infinite loop) if (this.dead) { if (!this.deathAnimStarted) { this.deathAnimStarted = true; this._playAni("death", 0); // If available, disable looping this.sprite.ani?.noLoop?.(); } else { this._setAni("death"); // Hard clamp to last frame (works even if noLoop doesn't exist) const def = this.assets?.playerAnis?.death; const frames = Number(def?.frames ?? 1); if (this.sprite.ani) this.sprite.ani.frame = Math.max(0, frames - 1); } return; } if (won) { // optional: choose idle when winning this._setAni("idle"); return; } if (this.knockTimer > 0 || this.pendingDeath) { this._setAniFrame("hurtPose", 1); return; } // attack animation is started by startAttack(); don't stomp it here if (this.attacking) return; if (!grounded) { const f = this.sprite.vel.y < 0 ? 0 : 1; this._setAniFrame("jump", f); return; } const moving = Math.abs(this.sprite.vel.x) > 0.01; this._setAni(moving ? "run" : "idle"); } // ---------- // damage / effects // ---------- takeDamageFromX(sourceX) { if (this.invulnTimer > 0 || this.dead) return false; this.health = Math.max(0, this.health - 1); if (this.health <= 0) this.pendingDeath = true; this.invulnTimer = this.INVULN_FRAMES; this.knockTimer = this.KNOCK_FRAMES; const dir = this.sprite.x < sourceX ? -1 : 1; this.sprite.vel.x = dir * this.KNOCKBACK_X; this.sprite.vel.y = -this.KNOCKBACK_Y; // cancel attack this.attacking = false; this.attackFrameCounter = 0; this.attackHitThisSwing = false; return true; } applyHurtBlinkTint() { if (!this.sprite) return; if (!this.dead && this.invulnTimer > 0) { this.sprite.tint = Math.floor(this.invulnTimer / 4) % 2 === 0 ? "#ff5050" : "#ffffff"; } else { this.sprite.tint = "#ffffff"; } } }

Response:

Ah! Thanks, now I can see exactly how your `PlayerEntity` is structured. The issue where the **body appears on top and the head is at the bottom** is almost certainly because of **`this.ANI_OFFSET_Y`** and the sprite frame origin.

In your current code:

this.sprite.anis.offset.y = this.ANI_OFFSET_Y;

That offset determines **how the animation frames are drawn relative to the sprite's position**. If `ANI_OFFSET_Y` is positive, it will push the sprite **down**, which can make it look like the body is above the head.

Quick Fix

1. Make `ANI_OFFSET_Y` **negative**, so the sprite shifts **up** to align the head at the top:

```
this.ANI_OFFSET_Y = - (frameH / 2); // move the sprite up by half its height
```

2. Or, if you want fine control, you can tweak it until it looks right:

```
this.sprite.anis.offset.y = -4; // trial and error
```

3. Optional: make sure the collider matches the visible sprite:

```
this.sprite.w = this.COLLIDER_W;
this.sprite.h = this.COLLIDER_H;
// Center the collider on the sprite
this.sprite.offset.x = 0;
this.sprite.offset.y = 0;
```

Debug Tip

Draw the sprite's origin as a dot to see where it's anchored:

```
fill("red");
circle(this.sprite.x, this.sprite.y, 4);
```

That will help you adjust `ANI_OFFSET_Y` so the head lines up correctly.

If you want, I can rewrite your **`buildSprites()`** method so the sprite is perfectly upright, the collider is aligned, and the ground sensor works—basically a drop-in version that fixes the "head below body" issue.

Do you want me to do that?

Prompt:
fixed the sprite idle but when I run or jump, I can only see half the body and top frame of sprite container shows part of bottom half of body

Response:

This is a **sprite sheet frame clipping issue**, not a physics or collider problem. Your idle works because that animation probably has the correct frame size, but the **run/jump animations are taller than the frame size you're telling p5play**.

Right now you force every animation frame to:

frameW = 32
frameH = 32

But if your run/jump sprites are **taller than 32px**, p5play will **crop the image**, which is why:

- you see **half the body**
- the **top of the frame shows the bottom of the sprite**

Fix 1 (Most Likely): Increase frame height

If your sprite frames are actually **48px or 64px tall**, change:

```
const { frameW = 32, frameH = 32 } = this.tilesCfg;
```

to something like:

```
const { frameW = 32, frameH = 48 } = this.tilesCfg;
```

or

```
const { frameW = 32, frameH = 64 } = this.tilesCfg;
```

Then keep:

```
this.sprite.anis.w = frameW;
this.sprite.anis.h = frameH;
```

Fix 2: Different frame size for animations

If idle is 32x32 but run/jump are larger, set the **largest size**:

```
this.sprite.anis.w = 32;
this.sprite.anis.h = 48;
```

p5play needs the **full frame size** or it will crop.

Fix 3 (If frames are centered differently)

After fixing height, you may need a small offset tweak:

this.sprite.anis.offset.y = -8;

But **do this only after fixing frame height**.

Quick way to confirm the problem

Open the sprite sheet and check:

- width of **one frame**
- height of **one frame**

Example:

Animation	Frame Size
idle	32x32
run	**32x48**
jump	**32x48**

If that's the case, your game must use **32x48**, not 32x32.

✅ **90% chance your fix is simply:**

const { frameW = 32, frameH = 48 } = this.tilesCfg;

If you want, send **one screenshot of the sprite sheet**, and I can tell you the **exact frame width/height** you should use so the clipping disappears.

Prompt:
Uploaded image

Response:
Thanks for the sprite sheet — that helps a lot. 👍

Your image size is **258 × 204 px**. That strongly suggests the frames are arranged like this:

- **8 columns**
- **6 rows**

- Each frame ≈ **32 × 34 px** (not 32 × 32)

Why:

- 8 × 32 ≈ **256** (matches the width with a tiny margin)
- 6 × 34 = **204** (exactly your height)

So the **actual frame height is 34**, not 32. When you set:

```
this.sprite.anis.w = 32;
this.sprite.anis.h = 32;
```

p5play crops each frame to **32px tall**, which cuts off the bottom of the character during **run/jump**, making the body look split.

✅ Fix

Change your animation frame height to **34**.

```
this.sprite.anis.w = 32;
this.sprite.anis.h = 34;
```

Keep the collider the same if you want tighter physics:

```
this.sprite.w = this.COLLIDER_W;
this.sprite.h = this.COLLIDER_H;
```

Optional: adjust vertical alignment

Because the sprite is now slightly taller, you may want a small offset:

```
this.sprite.anis.offset.y = -1;
```

or

```
this.sprite.anis.offset.y = -2;
```

Just tweak until the feet sit correctly on the ground.

Why idle looked fine

Idle probably looked correct because that animation frame didn't extend to the very bottom of the 34px cell, so the 32px crop wasn't obvious.

Run/jump extend further downward → **visible cropping**.

✅ **Final working block:**

```
this.sprite.anis.w = 32;
this.sprite.anis.h = 34;
this.sprite.anis.offset.y = -2;
this.sprite.addAnis(anis);
this._setAni("idle");
```

💡 If you'd like, I can also show you a **much cleaner way to load this sheet in p5play** so you **never have to guess frame sizes again**, and it will automatically detect the grid

Prompt:

// src/entities/PlayerEntity.js // Player entity (WORLD entity). // // Responsibilities: // - Own player state (health/timers/flags) // - Own player sprites (main sprite + ground sensor) // - Provide small action methods (move/jump/attack/damage) // - Apply animation based on current state (visual state driven by world state) // // Non-goals: // - Does NOT read input directly (PlayerController does) // - Does NOT decide game win state (Level does) export class PlayerEntity { constructor(pkg, assets) { this.pkg = pkg; this.assets = assets; this.tuning = pkg.tuning || {}; this.tilesCfg = pkg.tiles || {}; this.bounds = pkg.bounds || {}; this.levelData = pkg.level || {}; // sprites this.sprite = null; this.sensor = null; // spawn const ps = this.levelData.playerStart || { x: this.tilesCfg.frameW ?? 32, y: (this.bounds.levelH ?? 0) - (this.tilesCfg.tileH ?? 24) * 4, }; this.startX = ps.x; this.startY = ps.y; // stats this.maxHealth = Number(this.tuning.player?.maxHealth ?? 3); this.health = this.maxHealth; // state this.dead = false; this.pendingDeath = false; // death animation latch (prevents looping forever) this.deathAnimStarted = false; // timers this.invulnTimer = 0; this.knockTimer = 0; // attack this.attacking = false; this.attackFrameCounter = 0; this.attackHitThisSwing = false; // tuning defaults this.MOVE_SPEED = Number(this.tuning.player?.moveSpeed ?? 1.5); this.JUMP_STRENGTH = Number(this.tuning.player?.jumpStrength ?? 4.5); this.INVULN_FRAMES = Number(this.tuning.player?.invulnFrames ?? 45); this.KNOCK_FRAMES = Number(this.tuning.player?.knockFrames ?? 30); this.KNOCKBACK_X = Number(this.tuning.player?.knockbackX ?? 2.0); this.KNOCKBACK_Y = Number(this.tuning.player?.knockbackY ?? 3.2); this.COLLIDER_W = Number(this.tuning.player?.w ?? 18); this.COLLIDER_H = Number(this.tuning.player?.h ?? 12); this.ANI_OFFSET_Y = Number(this.tuning.player?.aniOffsetY ?? -8); // attack window this.ATTACK_START = Number(this.tuning.player?.attackStartFrame ?? 4); this.ATTACK_END = Number(this.tuning.player?.attackEndFrame ?? 8); this.ATTACK_FINISH = Number(this.tuning.player?.attackFinishFrame ?? 12); } // ---------- // animation safety // ---------- _hasAni(name) { return !!(this.sprite?.anis && this.sprite.anis[name]); } _setAni(name) {

```
if (!this._hasAni(name)) return false; this.sprite.ani = name; return true; } _setAniFrame(name, frame) { if
(!this._setAni(name)) return false; if (this.sprite.ani) this.sprite.ani.frame = frame; return true; }
_playAni(name, startFrame = 0) { if (!this._setAni(name)) return false; if (this.sprite.ani) {
this.sprite.ani.frame = startFrame; this.sprite.ani.play?.(); } return true; } // ---------------------- // build / reset
// ---------------------- buildSprites() { const { frameW = 30, frameH = 64 } = this.tilesCfg; this.sprite = new
Sprite(this.startX, this.startY, frameW, frameH); this.sprite.rotationLock = true; const anis =
this.assets?.playerAnis; const img = this.assets?.playerImg; if (img) this.sprite.spriteSheet = img; if (anis
&& typeof anis === "object") { this.sprite.anis.w = frameW; this.sprite.anis.h = frameH;
this.sprite.anis.offset.y = -9; this.sprite.addAnis(anis); this._setAni("idle"); } else { this.sprite.img = img; } //
collider this.sprite.w = this.COLLIDER_W; this.sprite.h = this.COLLIDER_H; this.sprite.friction = 0;
this.sprite.bounciness = 0; // ground sensor (query-only) this.sensor = new Sprite(); this.sensor.x =
this.sprite.x; this.sensor.y = this.sprite.y + this.sprite.h / 2; this.sensor.w = this.sprite.w; this.sensor.h = 2;
this.sensor.mass = 0.01; this.sensor.removeColliders(); this.sensor.visible = false; const j = new
GlueJoint(this.sprite, this.sensor); j.visible = false; return this; } reset() { this.health = this.maxHealth;
this.dead = false; this.pendingDeath = false; this.deathAnimStarted = false; this.invulnTimer = 0;
this.knockTimer = 0; this.attacking = false; this.attackFrameCounter = 0; this.attackHitThisSwing = false; if
(!this.sprite) return; this.sprite.x = this.startX; this.sprite.y = this.startY; this.sprite.vel.x = 0; this.sprite.vel.y
= 0; this.sprite.tint = "#ffffff"; this._setAni("idle"); } // ---------------------- // queries // ----------------------
isGrounded(solids) { const s = this.sensor; if (!s) return false; // Allow either { ground, platformsL, ... } OR
an array [ground, platformsL, ...] const list = Array.isArray(solids) ? solids : Object.values(solids || {}); for
(const g of list) { if (g && s.overlapping(g)) return true; } return false; } // ---------------------- // timers //
---------------------- tickTimers() { if (this.invulnTimer > 0) this.invulnTimer--; if (this.knockTimer > 0)
this.knockTimer--; } // ---------------------- // actions (mutations) // ---------------------- stopX() { this.sprite.vel.x
= 0; } moveLeft() { this.sprite.vel.x = -this.MOVE_SPEED; this.sprite.mirror.x = true; } moveRight() {
this.sprite.vel.x = this.MOVE_SPEED; this.sprite.mirror.x = false; } jump() { this.sprite.vel.y = -1 *
this.JUMP_STRENGTH; } startAttack() { this.attacking = true; this.attackHitThisSwing = false;
this.attackFrameCounter = 0; this.stopX(); this._playAni("attack", 0); } markAttackHit() {
this.attackHitThisSwing = true; } clampToBounds(bounds) { const half = (this.sprite?.w ?? 0) / 2; const
maxX = (bounds?.levelW ?? this.sprite.x) - half; this.sprite.x = constrain(this.sprite.x, half, maxX); } //
---------------------- // animation (visual state) // ---------------------- applyAnimation({ grounded, won }) { if
(!this.sprite?.anis || Object.keys(this.sprite.anis).length === 0) return; // ---- DEAD: play once, then hold
last frame (prevents infinite loop) if (this.dead) { if (!this.deathAnimStarted) { this.deathAnimStarted = true;
this._playAni("death", 0); // If available, disable looping this.sprite.ani?.noLoop?.(); } else {
this._setAni("death"); // Hard clamp to last frame (works even if noLoop doesn't exist) const def =
this.assets?.playerAnis?.death; const frames = Number(def?.frames ?? 1); if (this.sprite.ani)
this.sprite.ani.frame = Math.max(0, frames - 1); } return; } if (won) { // optional: choose idle when winning
this._setAni("idle"); return; } if (this.knockTimer > 0 || this.pendingDeath) { this._setAniFrame("hurtPose",
1); return; } // attack animation is started by startAttack(); don't stomp it here if (this.attacking) return; if
(!grounded) { const f = this.sprite.vel.y < 0 ? 0 : 1; this._setAniFrame("jump", f); return; } const moving =
Math.abs(this.sprite.vel.x) > 0.01; this._setAni(moving ? "run" : "idle"); } // ---------------------- // damage /
effects // ---------------------- takeDamageFromX(sourceX) { if (this.invulnTimer > 0 || this.dead) return false;
this.health = Math.max(0, this.health - 1); if (this.health <= 0) this.pendingDeath = true; this.invulnTimer =
this.INVULN_FRAMES; this.knockTimer = this.KNOCK_FRAMES; const dir = this.sprite.x < sourceX ? -1
: 1; this.sprite.vel.x = dir * this.KNOCKBACK_X; this.sprite.vel.y = -this.KNOCKBACK_Y; // cancel attack
this.attacking = false; this.attackFrameCounter = 0; this.attackHitThisSwing = false; return true; }
applyHurtBlinkTint() { if (!this.sprite) return; if (!this.dead && this.invulnTimer > 0) { this.sprite.tint =
Math.floor(this.invulnTimer / 4) % 2 === 0 ? "#ff5050" : "#ffffff"; } else { this.sprite.tint = "#ffffff"; } } }
```

Response:

The problem is here in your `buildSprites()`:

const { frameW = 30, frameH = 64 } = this.tilesCfg;

But the sprite sheet you showed earlier uses **32 × 34 frames**.

Right now you are telling p5play each frame is **30 × 64**, which causes:

- wrong cropping
- half the body disappearing
- the next row of the sheet leaking into the frame

✅ Correct Fix

Change this line:

const { frameW = 30, frameH = 64 } = this.tilesCfg;

to:

const { frameW = 32, frameH = 34 } = this.tilesCfg;

Then fix the animation setup

Replace this block:

```
this.sprite.anis.w = frameW;
this.sprite.anis.h = frameH;
this.sprite.anis.offset.y = -9;
```

with:

```
this.sprite.anis.w = 32;
this.sprite.anis.h = 34;
this.sprite.anis.offset.y = -2;
```

Final working `buildSprites()` section

```
buildSprites() {
 const { frameW = 32, frameH = 34 } = this.tilesCfg;

 this.sprite = new Sprite(this.startX, this.startY, frameW, frameH);
 this.sprite.rotationLock = true;

 const anis = this.assets?.playerAnis;
 const img = this.assets?.playerImg;

 if (img) this.sprite.spriteSheet = img;

 if (anis && typeof anis === "object") {
   this.sprite.anis.w = 32;
   this.sprite.anis.h = 34;
   this.sprite.anis.offset.y = -2;

   this.sprite.addAnis(anis);
   this._setAni("idle");
 } else {
   this.sprite.img = img;
 }
```

Why this fixes everything

Your sprite sheet grid is:

columns	rows	frame size
8	6	**32 × 34**

Using the wrong frame size causes p5play to read frames like this:

[half frame]
[next row]

which is exactly the glitch you described.

Optional (recommended improvement)

Your collider is much smaller than the sprite:

this.COLLIDER_W = 18
this.COLLIDER_H = 12

That's fine, but you may want the sprite centered better:

this.sprite.anis.offset.y = -4;

✅ If you want, I can also show you a **much cleaner p5play setup for sprite sheets** that will:

- auto-detect frames
- fix alignment automatically
- reduce ~10 lines of code here

and make future sprites easier to add.

how to add a debug screen

Adding a **debug screen in p5play** is actually super easy—and really useful for things like your collider vs sprite issues.

✅ 1. Turn on built-in debug (fastest way)

Just add this in your `draw()`:

allSprites.debug = true;

What this shows

- ✅ collider boxes (green outlines)
- ✅ sprite centers
- ✅ sensor overlaps
- ✅ group boundaries

This will instantly show you **why things are getting cut or misaligned**.

✅ 2. Toggle debug with a key (recommended)

So it's not always on:

```
let debugMode = false;

function keyPressed() {
 if (key === 'd') debugMode = !debugMode;
}

function draw() {
 allSprites.debug = debugMode;
}
```

✅ 3. Add a simple debug HUD (text on screen)

You can display useful info like position, velocity, etc.

```
function draw() {
 background(0);

 fill(255);
 textSize(14);

 text(`X: ${player.sprite.x}`, 10, 20);
 text(`Y: ${player.sprite.y}`, 10, 40);
 text(`VelX: ${player.sprite.vel.x}`, 10, 60);
 text(`VelY: ${player.sprite.vel.y}`, 10, 80);
}
```

✅ 4. Highlight a specific group (like frogs)

```
frogGroup.debug = true;
```

or inside your controller:

```
for (const s of this.group) {
 s.debug = true;
```

}

🧠 What to look for (for your current issue)

Turn debug on and check:

1. Is the collider tiny?

You set:

```
g.w = 10;
g.h = 6;
```

→ You'll see a tiny box under a big frog

---

2. Is the sprite offset wrong?

```
g.anis.offset.y = -4;
```

→ Debug will show sprite vs collider mismatch

---

3. Is the sprite being cropped?

If the green box is fine but the frog is cut → it's a **sprite sheet slicing issue**, not physics.

🚀 Pro Tip

You can also slow the game to inspect:

```
world.timeScale = 0.2;
```

If you want

I can help you build a **proper debug panel (like Unity)** that shows:

- grounded state
- current animation
- attack frames
- health

Super useful for your assignment 👀

Prompt:
Where is my draw?

Response:

Good question — in your setup, you likely **don't have a visible `draw()` in the same file** because your project is structured into modules (PlayerEntity, Controller, Level, etc.).

But **p5 / p5play ALWAYS has a `draw()` somewhere** — that's the main game loop.

🔍 Where to find your `draw()`

Search your project for:

function draw(

or

draw() {

It's usually in one of these files:

- `sketch.js` ✅ (most common)
- `main.js`
- `Game.js`
- `World.js`
- `Level.js`

👀 What it probably looks like

Something like:

```
function draw() {
 background(0);

 // update game
 level.update();
}
```

or

```
function draw() {
 world.step();
}
```

🧠 Why you couldn’t find it

Your `PlayerEntity.js` is **not responsible for drawing** — it only defines behavior.

So your structure is likely:

```
sketch.js → runs draw()
  ↓
Level / GameWorld → updates everything
  ↓
PlayerEntity / LeafController → logic only
```

✅ Where to put debug

Once you find `draw()`, add:

```
allSprites.debug = true;
```

🚀 If you STILL can't find it

Search for:

```
new Canvas(
```

or

```
new World(
```

That file is usually the **main entry point**, and `draw()` will be there or nearby.

💡 Quick shortcut

In VS Code:

👉 Press:

Ctrl + Shift + F

Search:

draw(

Prompt:
found draw in main.js, now tell me how I can make a debug screen

Response:

✅ Step 1: Confirm DebugOverlay is being created

You already have this in `initRuntime()`:

```
debugOverlay = new DebugOverlay();
```

and passed into Game:

```
game = new Game(levelPkg, assets, {
 hudGfx,
 inputManager,
 soundManager,
 debugOverlay,
});
```

✅ Good — no change needed here.

✅ Step 2: Draw the debug overlay (THIS is what you're missing)

Inside your `draw()` function, after `game.draw(...)`, add:

```
debugOverlay?.draw({
 game,
 camera,
});
```

So your draw flow becomes:

```
game.draw({
 drawHudFn: () => {
   camera.off();
   try {
     drawingContext.imageSmoothingEnabled = false;
     imageMode(CORNER);
     image(hudGfx, 0, 0);
   } finally {
     camera.on();
     noTint();
   }
 },
});

// 👇 ADD THIS
debugOverlay?.draw({
 game,
 camera,
});
```

✅ Step 3: Add actual debug info (inside DebugOverlay.js)

Open `DebugOverlay.js` and make sure it actually draws something.

Here’s a simple working version:

```
export class DebugOverlay {
 constructor() {
   this.enabled = true;
```

```
  }

  draw({ game, camera }) {
    if (!this.enabled) return;

    camera.off(); // draw in screen space

    push();
    fill(255);
    textSize(12);
    textAlign(LEFT, TOP);

    const player = game?.level?.playerCtrl?.sprite;

    let y = 10;

    text(`FPS: ${Math.round(frameRate())}`, 10, y); y += 15;

    if (player) {
      text(`Player X: ${player.x.toFixed(1)}`, 10, y); y += 15;
      text(`Player Y: ${player.y.toFixed(1)}`, 10, y); y += 15;
      text(`Vel X: ${player.vel.x.toFixed(2)}`, 10, y); y += 15;
      text(`Vel Y: ${player.vel.y.toFixed(2)}`, 10, y); y += 15;
    }

    pop();

    camera.on();
  }
}
```

Prompt:
How do I add sounds / where can I put them in my code?

Response:

✅ STEP 1 — Load sounds in `boot()`

Right now you created `soundManager` but didn't load anything.

👉 Add this **inside `boot()` RIGHT AFTER:**

```
soundManager = new SoundManager();
```

✅ Add this:

```
soundManager.load("jump", "assets/sfx/jump.wav");
```

```
soundManager.load("hitEnemy", "assets/sfx/hitEnemy.wav");
soundManager.load("leafCollect", "assets/sfx/leafCollect.wav");
soundManager.load("receiveDamage", "assets/sfx/receiveDamage.wav");
soundManager.load("music", "assets/sfx/music.wav");
```

✅ STEP 2 — Start background music (loop + volume)

Still inside `boot()` (after loading sounds):

```
const music = soundManager.sfx["music"];
music.setLoop(true);
music.setVolume(0.2);
```

❗ DO NOT call `play()` yet (browser will block it)

✅ STEP 3 — Start music after user interaction

You already have this 👇 (good):

```
function mousePressed() {
 unlockAudioOnce();
}
```

🔥 Modify it to:

```
function mousePressed() {
 unlockAudioOnce();

 const music = soundManager?.sfx["music"];
 if (music && !music.isPlaying()) {
   music.play();
 }
}
```

👉 Same for `keyPressed`:

```
function keyPressed(evt) {
```

```
 unlockAudioOnce();

 const music = soundManager?.sfx["music"];
 if (music && !music.isPlaying()) {
   music.play();
 }

 if (evt.key === "x") {
   debugOverlay.enabled = !debugOverlay.enabled;
 }
 return preventKeysThatScroll(evt);
}
```

✅ STEP 4 — Connect gameplay → sounds

Now the important part: **WHEN sounds play**

You should NOT call sound directly inside entities.

👉 You already pass `soundManager` into `Game`:

```
game = new Game(levelPkg, assets, {
 hudGfx,
 inputManager,
 soundManager,
 debugOverlay,
});
```

🔥 BEST PRACTICE (use EventBus)

Inside your `Game.js`, add:

```
this.events.on("player:jump", () => {
 this.soundManager.play("jump");
});

this.events.on("leaf:collected", () => {
 this.soundManager.play("leafCollect");
});

this.events.on("player:damaged", () => {
```

```
 this.soundManager.play("receiveDamage");
});

this.events.on("enemy:hit", () => {
 this.soundManager.play("hitEnemy");
});
```

✅ STEP 5 — Emit events from gameplay

Now inside your entities:

Example: Player jump

```
this.events.emit("player:jump");
```

Example: Leaf collected

```
this.events.emit("leaf:collected");
```

Example: Damage

```
this.events.emit("player:damaged");
```